Renode is the ideal emulator for testing **NovaOS v0.1** because it includes a native **STM32H743** platform definition (platforms/cpus/stm32h743.repl) that maps the Cortex-M7 core, Flash (0x08000000), DTCM-RAM (0x20000000), AXI-SRAM (0x24000000), STM32H7_RCC (0x58024400), GPIOB/GPIOD, and USART3 (0x40004800) at their exact hardware addresses. By contrast, mainline **QEMU** (qemu-system-arm) lacks a native stm32h743 machine model—its closest Cortex-M7 boards (mps2-an500 / mps2-an505) place Flash, SRAM, and UARTs at completely different physical addresses, which would force you to maintain a separate linker script and UART driver just for emulation. With Renode, you boot the **exact unmodified novaos_v01.elf** that you will flash to physical silicon.

1. Install Renode on Linux

On Debian/Ubuntu, grab the official portable package or .deb release from GitHub so you get the latest stm32h743.repl and STM32H7_RCC models:

```
Bash
# Download and install the latest Renode .deb package (Ubuntu/Debian)
wget
https://github.com/renode/renode/releases/latest/download/renode_latest_amd64.deb
sudo apt install ./renode_latest_amd64.deb

# Verify installation
renode --version
```

2. The One Bare-Metal "Gotcha" in Emulation (And How We Fix It)

When running real bare-metal firmware (rather than a high-level OS port that skips power-controller checks) inside an emulator, there are two hardware registers you must account for:

1. **LDO Voltage Regulator Ready Flag (PWR_CSR1_ACTVOSRDY)**: In kernel.c, our clock_init() routine enables the internal LDO regulator and polls PWR_CSR1 at 0x58024804 waiting for bit 13 (ACTVOSRDY, 0x00002000) to assert. Because Renode does not simulate analog LDO voltage settling, reading 0x58024804 returns 0x00000000 by default—which would cause clock_init() to spin forever. We solve this in one line inside our Renode script using sysbus Tag to return 0x00002000 (ACTVOSRDY = 1) whenever 0x58024804 is read.
2. **SysTick Clock Frequency Match**: Renode's stm32h743.repl defaults the nvic SysTick clock to 480_000_000 Hz (480 MHz). Because Phase 1 runs on the default 64 MHz HSI oscillator prior to our Phase 2 PLL synthesis, we set sysbus.nvic Frequency 64000000 in our script so ticks and uptime run in exact real time.

3. Create the Renode Script (nova32_phase1.resc)

Save this file as nova32_phase1.resc in your project root directory alongside your Makefile:

```
Python
# ============================================================================
# Project NOVA-32 | Phase 1 Renode Emulation Script (nova32_phase1.resc)
# Target: STMicroelectronics STM32H743ZIT6 (Cortex-M7)
# ============================================================================

using sysbus
$name?="NOVA-32"
mach create $name

# 1. Load the built-in STM32H743 platform description
machine LoadPlatformDescription @platforms/cpus/stm32h743.repl

# 2. Match SysTick frequency to Phase 1 clock (64 MHz HSI)
sysbus.nvic Frequency 64000000

# 3. Mock PWR_CSR1 (0x58024804) bit 13 (ACTVOSRDY = 0x2000) so the bare-metal
#    LDO voltage scaling readiness loop in clock_init() completes immediately.
sysbus Tag <0x58024804 4> "PWR_CSR1" 0x00002000

# 4. Mock RCC_CR (0x58024400) bit 2 (HSIRDY = 0x00000004) + bit 0 (HSION = 0x1)
#    as a fallback in case your Renode version resets RCC_CR to 0.
sysbus Tag <0x58024400 4> "RCC_CR" 0x00000005

# 5. Expose USART3 both in a GUI analyzer window AND on localhost TCP port 3456
#    so you can attach with telnet/netcat from any Linux terminal.
showAnalyzer sysbus.usart3
emulation CreateServerSocketTerminal 3456 "nova32_cli_socket" false
connector Connect sysbus.usart3 nova32_cli_socket

# 6. Start a GDB server on port 3333 so `make debug` attaches seamlessly
machine StartGdbServer 3333

# 7. Define reset macro to reload ELF and set Vector Table Offset Register
macro reset
"""
    sysbus LoadELF @build/novaos_v01.elf
    sysbus.cpu VectorTableOffset 0x08000000
"""
```

```
runMacro $reset
echo "================================================================"
echo " NOVA-32 Phase 1 loaded!"
echo " - UART Console TCP Socket : telnet localhost 3456"
echo " - GDB Remote Debug Server : localhost:3333"
echo " Starting CPU execution..."
echo "================================================================"
start
```

4. Add Emulation Targets to Your Makefile

Append these two targets to the bottom of the Makefile from earlier so you can launch Renode in either desktop GUI mode or pure headless terminal mode:

```
Makefile
# Launch Renode with GUI UART window
emulate: $(BUILD_DIR)/$(TARGET).elf
    renode nova32_phase1.resc

# Launch Renode in pure headless terminal mode (connect via telnet localhost 3456)
emulate-headless: $(BUILD_DIR)/$(TARGET).elf
    renode --disable-xwt --console nova32_phase1.resc
```

5. Running and Testing NovaOS v0.1

Option A: Desktop Mode (GUI Analyzer Window)

Run make emulate. Renode will compile build/novaos_v01.elf if needed, open the Renode Monitor and the sysbus.usart3 terminal window, and immediately boot into the NovaOS v0.1 banner and nova32> prompt.

Option B: Pure Linux Command-Line Mode (No GUI Required)

If you prefer working strictly in the terminal without X11/GUI windows:

1. In **Terminal 1**, start the headless emulator:
   ```
   Bash
   make emulate-headless
   ```

2. In **Terminal 2**, connect to the USART3 TCP socket on port 3456:
   ```
   Bash
   nc localhost 3456
   # or: telnet localhost 3456
   ```

   Press **Enter** and you will get the live interactive nova32> shell.

Commands to Test Inside the Emulated Shell

Once at the nova32> prompt, verify that the memory map, interrupts, heap allocator, and hardware registers work as designed:

- sysinfo — Verifies CPU ID (0x411FC271 for Cortex-M7 r1p1), vector table address, and 64 MHz clock reporting.
- ticks — Confirms the SysTick_Handler interrupt is firing 1,000 times per second and incrementing the volatile tick counter in DTCM-RAM.
- alloc 256 — Allocates 256 bytes from the 512 KB AXI-SRAM heap (0x24000000) and returns the pointer address (e.g., 0x24000008).
- poke 0x24000008 0xDEADBEEF — Writes a 32-bit word into your newly allocated AXI-SRAM block.
- peek 0x24000008 — Reads back 0xDEADBEEF from AXI-SRAM.
- hexdump 0x08000000 64 — Dumps the first 64 bytes of Bank 1 Flash, showing the initial stack pointer (0x20020000 in little-endian: 00 00 02 20) followed by the Reset_Handler vector.

6. Single-Stepping the Kernel with GDB

Because nova32_phase1.resc starts a GDB server on port 3333, you can attach arm-none-eabi-gdb (or gdb-multiarch) while Renode is running:

```bash
# In another terminal window:
make debug
```

Inside GDB, you can inspect the hardware state directly:

```
(gdb) break USART3_IRQHandler
(gdb) continue
# Type a character in the USART3 terminal window — GDB will halt inside the ISR!
(gdb) info registers
(gdb) x/16xw 0x20000000
```